

Report: Walkthrough of TryHackMe Room "IDS Evasion"

Submitted by : Godly K Mathews

1. Objective

The "IDS Evasion" room on TryHackMe is an educational module designed to teach the fundamental concepts of Intrusion Detection Systems (IDS) and the techniques attackers use to bypass them. The primary goal of this report is to document the key techniques learned in each task, explain *why* they work, and, most importantly, analyze the defensive measures and security principles that can be used to detect and mitigate these evasion attempts.

2. Task-by-Task Breakdown

Task 1-2: Introduction to IDS Basics

- **What was done:** This section served as a theoretical introduction to Intrusion Detection Systems.
- **Explanation & Concepts:**
 - **NIDS (Network-based IDS):** A NIDS monitors traffic on the network. It's like a security guard watching all the cars on the road. It's great for spotting known attacks as they travel across the network.
 - **HIDS (Host-based IDS):** A HIDS is installed on a specific device (a "host") and monitors its internal activity, such as file changes, process execution, and logs. This is like a security guard *inside* a building, watching for suspicious activity.
 - **Signature-based Detection:** This is the most common method. The IDS has a database of "signatures" (like fingerprints) for known attacks (e.g., a specific string in a malware-infected packet). It's very fast and accurate for known threats but completely blind to new, "zero-day" attacks.
- **Defensive Takeaway:** No single solution is enough. This task establishes the core concept of **Defense in Depth**—relying on multiple layers of security (both NIDS and HIDS) is essential.

Task 3-4: NIDS and Basic Reconnaissance Evasion

- **What was done:** This task involved running basic `nmap` (Network Mapper) scans to identify open ports and services, a standard first step in any attack. The key was to see how the NIDS (in this case, Suricata) reacted.
- **Evasion Technique:** A simple evasion technique was introduced: changing the `nmap` **User-Agent**. By default, `nmap`'s scanning engine identifies itself with a clear "Nmap" user agent, which is an easy signature for an IDS to block. By using a flag like `--script http-useragent --script-args http.useragent="Mozilla/5.0..."`, the scan is disguised as regular web browser traffic.

- **Defensive Takeaway:** Relying on simple, high-level signatures like a User-Agent string is a weak defense. A more robust NIDS configuration would use behavioral analysis, such as flagging an IP address that connects to 1,000 different ports in 10 seconds, as this is behavior characteristic of a scan, regardless of the user agent.

Task 5: Advanced Scanning Evasion (Nikto)

- **What was done:** The web vulnerability scanner `nikto` was used to find vulnerabilities on the target web server.
- **Evasion Technique:** The primary technique here was **timing manipulation**. A normal `nikto` scan is very fast and "loud," making it easy for an IDS to detect based on the high rate of requests. By using scan tuning flags (e.g., `-T` in `nmap` or similar timing options in other tools), an attacker can perform a "low-and-slow" scan. This involves sending one request, waiting several seconds, sending another, and so on.
- **Defensive Takeaway:** This technique is designed to defeat simple, rate-based IDS rules. The defense against this is a SIEM (Security Information and Event Management) system. A SIEM can aggregate logs over a long period. While a NIDS might not see "50 requests in 1 second," a SIEM can be configured to alert on "500 requests from one IP over 2 hours," which would catch this slower, more methodical attack.

Task 6-7: OSINT, Exploitation, and Ruleset Evasion

- **What was done:** This task chain demonstrated a realistic attack.
 1. **OSINT:** Using Google and other public resources, the version of the running Grafana service (8.2.5) was identified.
 2. **Vulnerability Research:** A public vulnerability (CVE-2021-43798) was found for this specific version. This CVE is a directory traversal vulnerability.
 3. **Exploitation:** The CVE was exploited to read the `/etc/shadow` file from the server.
- **Evasion Technique:** The evasion here was implicit. The attack worked because the default Suricata ruleset being used by the NIDS did not have a signature to detect this specific exploit.
- **Defensive Takeaway:** This highlights the most critical defensive principle: **Patch Management**. The vulnerability should have been patched. From an IDS perspective, it shows that an IDS is only as good as its rules. Security teams must ensure their IDS rules are constantly updated. They can also write custom rules to detect patterns (like `..%2f`) associated with directory traversal attacks, even if the specific CVE isn't in the default set.

Task 8-9: HIDS and Post-Exploitation Detection

- **What was done:** After gaining initial access, the focus shifted to privilege escalation. A common enumeration script, `linPEAS`, was downloaded and run on the target machine.

- **Detection:** This is where the HIDS (Wazuh) shined. The NIDS (Suricata) was blind to this activity, as it was all happening *on the host* itself. The HIDS, however, immediately generated alerts for several reasons:
 1. **File Integrity Monitoring (FIM):** An alert (e.g., severity 5) was triggered when a new, suspicious file (`linpeas.sh`) was written to the `/tmp` directory.
 2. **Process Monitoring:** The HIDS also monitors command execution and can flag suspicious processes or commands.
- **Defensive Takeaway:** This task was the perfect demonstration of *why* HIDS is necessary. The NIDS may miss the entry, but the HIDS is designed to catch the attacker's actions *after* they get in.

Task 10-11: Privilege Escalation and Persistence

- **What was done:** The `linPEAS` script revealed that the compromised user was part of the `docker` group. This is a common misconfiguration, as this group membership effectively grants root privileges. This was abused to gain a root shell.
- **Persistence Technique:** To maintain access, a simple backdoor was created using a Docker container that mounted the host's root file system (`/`) and was configured to launch a reverse shell.
- **Defensive Takeaway:** The primary defense here is not detection but **prevention**. This is a system hardening and configuration issue. The principle of **Least Privilege** dictates that a standard user account should *never* be in the `docker` group. Proper system hardening would have prevented this privilege escalation entirely. A HIDS could also be configured to monitor Docker API activity or new container creation as a potential indicator of compromise.

3. Conclusion

This room provided a comprehensive overview of the cat-and-mouse game between attackers and defenders. The key learning objective was that security is a layered system. An attacker can often bypass one layer (like a NIDS with default rules) by using simple techniques (like changing a user agent or slowing down a scan).

1110
100%
Hack Me

DashboardLearnPracticeCompete

Access Machine

Starting your machine... please wait!

Learn > Intrusion Detection



Intrusion Detection

Learn cyber evasion techniques and put them to the test against two IDS

60 min

10,381

Share your achievement

Start AttackBox

Save Room

296 Recommend

Options

Room completed (100%)

Target Machine Information

Title	Target IP Address	Expires		
DemoCTFFinal	Shown in 0min 56s	59min 56s	?	Add 1 hour
				Terminate

Task 1 Introduction

Have you ever completed a CTF and wondered, "Would I have been detected?". This room will serve as an introduction to the world of intrusion detection

Start Machine